Scaler par la parallélisation

J2 après-midi

Worktrees, sous-agents, et votre ticket en parallèle

Émargement de vendredi après-midi : signez en arrivant

`edusign.app/sc/HOW661XV`

Pourquoi paralléliser maintenant

Ce matin

La conception reste avec vous.
L'implémentation peut passer **on the loop**, sous le contrôle des tests et de `/goal`.

Donc cet après-midi

Un développeur qui conçoit peut alimenter **plusieurs agents**… mais tout ce qu'ils produisent, il faut le **relire**.

15 ×

les tokens d'un système multi-agents, comparé à un chat (Anthropic, 06/2025)

On parallélise ce qui se relit vite.

Worktrees

1 worktree = 1 agent = 1 tâche

Deux sessions, un seul dossier : non

« Worse than unsupported »

Plusieurs sessions `/implement` dans le même checkout, d'après un retour cité par Matt Pocock :

- un `commit --amend` atterrit sur le commit d'une autre session
- un stash disparaît
- des commits partent sur la mauvaise branche



pas isolés : dépendances installées · .env · ports · base locale · volumes Docker

→ un script de préparation, versionné dans le repo

Même historique, autre dossier, autre branche.

En natif dans Claude Code

```
claude --worktree export-csv     # ou : claude -w export-csv
#   dossier : .claude/worktrees/export-csv/
#   branche : worktree-export-csv
claude -w pagination             # 2e terminal : 2e agent, 2e branche

claude --worktree "#1234"        # part de la PR 1234
claude --worktree export-csv --resume   # y revenir plus tard
```

- **Base** : la branche par défaut du remote ; `worktree.baseRef: "head"` pour partir de votre HEAD local
- **À la sortie** : un worktree propre est supprimé (session sans nom) ; sinon, Claude demande
- Ajoutez `.claude/worktrees/` au `.gitignore`

Ce que git n'isole pas

Isolé

- les fichiers
- la branche
- l'index

Pas isolé

- **Absent** du checkout neuf : les dépendances (`npm ci`, `dotnet restore` à relancer), les fichiers ignorés comme `.env` → `.worktreeinclude` (syntaxe `.gitignore`)
- **Partagé** par la machine : les ports, la base locale, les volumes Docker → un port et un nom de projet Docker par worktree

Bénéfriches l'a scripté : le skill `/worktree` lance `scripts/create-worktree.sh`, qui crée le worktree puis l'amorce : `pnpm install`, build de `shared`, copie des `.env` d'exemple.

Créer un worktree, c'est une commande ; le rendre utilisable, c'est un script à versionner.

Quand ça paie, quand s'en passer

Ça paie

- des tickets indépendants : des tranches verticales
- un agent long en arrière-plan pendant que vous faites autre chose
- comparer deux approches du même ticket
- essayer un refactoring risqué

S'en passer

- une tâche de quelques minutes
- deux tickets qui touchent les mêmes fichiers ou le même schéma de base
- un refactoring mécanique large : l'expand d'abord, seul

La limite, c'est votre relecture et votre attention : commencez à deux.

Sources : Bedard et al. (BCG), « When Using AI Leads to "Brain Fry" », HBR, 05/03/2026 · Simon Willison, 02/04/2026

EXERCICE À vous : préparez vos worktrees

1. **Committez** sur votre branche ce que ce matin a produit : le `.scratch/` de votre ticket, `GLOSSARY.md`, `docs/adr/` s'il existe, et le code de votre loop.
2. **Vérifiez** : `git status` est propre, et `git log -1` montre ce commit.
3. **Créez votre premier worktree** : `claude -w <ticket> --model sonnet`, vérifiez qu'il voit vos tickets (`!ls .scratch`), puis `/exit`. Ne lancez pas encore d'agent.

Dans Tailspin, vos worktrees partent de votre commit, et rien n'est à installer dedans.

Loop de midi en échec : `git fetch origin`, puis `git switch secours/07-filtres-01-fait`.
Projet perso : `baseRef` dans `.claude/settings.local.json`, et `.claude/worktrees/` dans le `.gitignore`.

Sous-agents

Un contexte frais, un résumé en retour

Ce que voit un sous-agent



Source : documentation Claude Code, page « sub-agents »

Le vôtre, et son cycle de vie

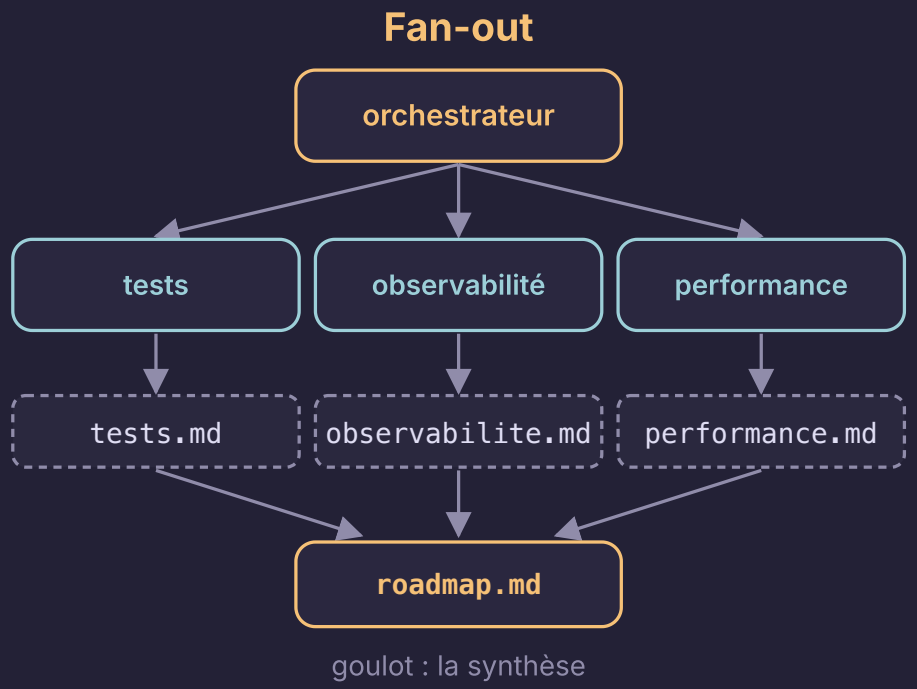
```
---
name: security-reviewer
description: Proactive security vulnerability
  scanner. Use AUTOMATICALLY after writing
  code that handles user input, …
model: opus
tools: [Bash, Grep, Glob, Read]
---
```

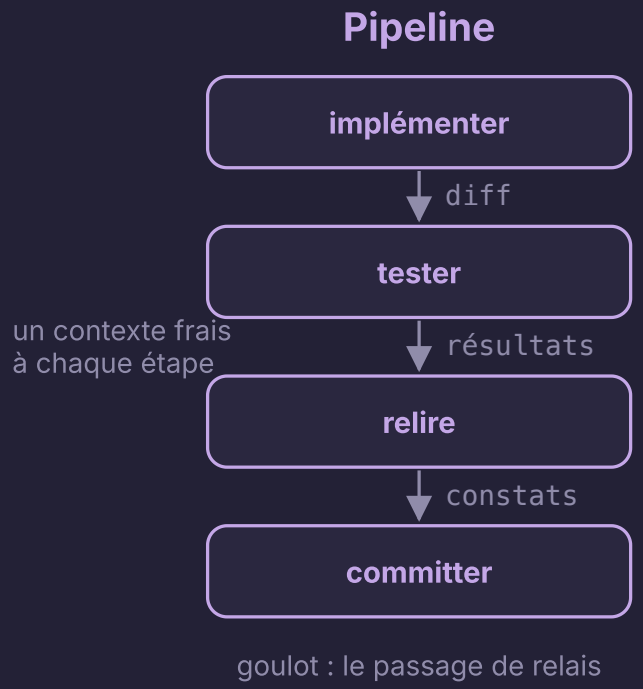
1. **Lancé** par Claude quand sa `description` correspond, ou par vous : « utilise un sous-agent pour… »
2. **En arrière-plan** par défaut ; `/tasks` pour suivre
3. **Le résultat** revient comme une notification
4. **Relancé** avec SendMessage, il garde son historique

`isolation: worktree` : son propre worktree · par défaut, 20 sous-agents en même temps au plus, 3 niveaux d'imbrication

Source : Bénéfriches (licence MIT), .claude/agents/security-reviewer.md, en-tête remis en forme

Deux façons de les assembler





Fan-out : des tâches indépendantes, souvent en lecture ; le goulot, c'est la synthèse.

Pipeline : des étapes qui s'enchaînent ; le goulot, c'est le passage de relais.

Un fichier sur chaque flèche.

Un pipeline, en un prompt

```
Prends le ticket .scratch/<fonctionnalité>/issues/01-….md.
Attends le rapport de chaque sous-agent avant l'étape suivante.
1. Implémentation : un sous-agent Sonnet nommé impl. Il ne lance
   pas d'agent. Il lance les tests ciblés avant son rapport ;
   un critère d'acceptation = une preuve.
2. Revue : un sous-agent Opus, en lecture seule, relit le diff
   contre le ticket, sans le rapport d'impl. Il ne corrige rien
   et ne lance pas d'agent. Il ne signale que ce qui touche la
   correction ou le ticket : bloquant / important / mineur,
   avec fichier:ligne.
3. Correction : renvoie les constats à impl avec SendMessage,
   un tour au plus.
4. Ne committe pas. Termine par : fait / reste à faire.
```

Prêts à l'emploi : `/implement` de Matt Pocock · `/code-review`, intégré à Claude Code

Un modèle fort pour juger, un modèle rapide pour exécuter.

Ce qui casse

Piège	Parade
Des rôles : un agent front, un agent back, un agent QA	découper par contexte, pas par personnage
La cascade : un sous-agent qui lance les siens, jusqu'à 3 niveaux	« Il ne lance pas d'agent » dans le brief ; pour une garantie, `tools` sans Agent, ou `CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH=1`
Le rapport qui ment	un chemin par affirmation ; vous en vérifiez une
Le coût	chaque sous-agent paie ses tokens, et son rapport entre dans votre fenêtre

Un sous-agent sert à protéger un contexte, pas à jouer un rôle.

Sources : Dex Horthy, « No Vibes Allowed », AI Engineer Code Summit, 11/2025 · documentation Claude Code, page « sub-agents » · github.com/mattpocock/skills, issue #573

Découper : par feature, couche, domaine ou étape

Découpage	Pour quoi faire	Avec quoi
Par feature, en tranche verticale	construire en parallèle	un worktree par ticket
Par couche : front, API, base	**à éviter pour paralléliser**	—
Par domaine : sécurité, tests, observabilité, performance	analyser en parallèle	des sous-agents en lecture seule
Par étape	enchaîner	un pipeline de sous-agents

Par couche : 26 tickets, ~20 passages d'agent par ticket fermé, dont trois sur quatre pour refaire.

Source : Matt Pocock, documentation de /to-tickets (github.com/mattpocock/skills)

Le graphe décide



01 fermé → 02 et 03 sont sur la frontière : en parallèle, un worktree chacun

- **Qui coordonne ?** Claude : des sous-agents · vous : des sessions · un script : des workflows dynamiques
- **Se parlent-ils ?** Sous-agents : via l'appelant · sessions : messagerie · équipe d'agents : directement
- **Touchent-ils les mêmes fichiers ?** Oui : un worktree chacun

Ce n'est pas vous qui décidez combien d'agents : c'est le graphe, et votre relecture.

ATELIER 3

Vos tickets, en parallèle

Le livrable

Deux tickets faits en même temps, puis fusionnés ; une analyse en parallèle ; vos tentations, classées.

Le déroulé

84 minutes, pause comprise, seul, chacun ses tickets. On avance **ensemble, étape par étape** : chaque étape s'affiche quand on y arrive.

Tant que le harness ne remonte rien, vous n'intervenez pas : vous notez chaque tentation, et sa raison, dans le cahier.

Votre graphe sous les yeux : on commence.

ATELIER 3 · ÉTAPE 1/5

Deux tickets, deux worktrees, en même temps

Terminal 1 : le skill de Pocock

```
claude -w <ticket-1> --model sonnet
/implement <chemin du ticket 1>
```

Terminal 2 : votre pipeline (slide 13, dans le cahier)

```
claude -w <ticket-2> --model sonnet
<le prompt du pipeline, ticket 2>
```

- ▶▶ **auto mode on** partout, et **lancez les deux** avant d'en regarder un.
- À regarder : le test d'abord, les sous-agents de revue (`/tasks`), le commit de `/implement` ; le « fait / reste à faire » du pipeline.

Un seul ticket sur la frontière ? Le pipeline seul. Fini en avance ? Votre condition `/goal`, dans chaque session.

11 minutes

ATELIER 3 · ÉTAPE 2/5

Relire, contre le ticket

```
!git show --stat            # /implement : il a committé
!git add -N . && git diff   # le pipeline : rien n'est committé
```

- **Le ticket ouvert à côté**, critère par critère : qu'est-ce qui le prouve ? Le diff et le test, pas le rapport de l'agent.
- **Le site** : « Lance le serveur de dev et donne-moi l'URL », dans chaque session ; un port chacun.
- **Le travail du pipeline** que vous gardez : « Committe ce ticket sur cette branche. »

Fini en avance ? Vérifiez dans le code un constat du relecteur Opus. Et lequel a trouvé le plus : la revue de `/implement`, ou le pipeline ?

12 minutes

ATELIER 3 · ÉTAPE 3/5

Fusionner : la frontière avance

```
# Terminal 3, à la racine, sur votre branche de travail
git merge --no-edit worktree-<ticket-1>
git merge --no-edit worktree-<ticket-2>
npm run test:unit
```

- **Un conflit ?** Dans ce terminal, `claude`, puis « Résous le conflit, et lance les tests ». Vous relisez.
- **Le ticket suivant est débloqué** : écrivez sa condition `/goal` (le modèle de ce matin), puis, dans un **terminal 4**, `claude -w <ticket-3> --model sonnet`, mode Auto vérifié, et lancez-la.

Rien de débloqué ? Une condition `/goal` sur ce qui reste de votre graphe.

13 minutes · la loop tourne pendant la pause

Pause

Votre loop tourne · reprise dans 15 minutes

ATELIER 3 · ÉTAPE 4/5

Analyser en parallèle

1. **Le verdict de votre loop** (terminal 4) : `/goal`, puis Ctrl+O. Atteinte : `!git add -N . && git diff`, puis « Committe ce ticket sur cette branche ».
2. **Terminal 3**, à la racine : `git merge --no-edit worktree-<ticket-3>` si elle est atteinte, puis `claude --model sonnet` et le prompt de l'analyse parallèle du cahier :

```
Audite le module src sur trois axes, en parallèle : tests,
accessibilité, sécurité. Lance trois sous-agents Explore […]
```

3. **Vérifiez vous-même** une affirmation de `roadmap.md` que l'orchestrateur n'a pas vérifiée.

Projet perso : votre module, vos axes. Fini en avance ? Un quatrième axe : performance.

17 minutes

ATELIER 3 · ÉTAPE 5/5

Classer vos tentations

Ma tentation	Ce qui a cloché	Ce que je corrige
« Le relancer : il a modifié un test »	**harness**	« aucun test n'est modifié » dans la condition, ou un hook
« Lui expliquer le format de date attendu »	**spec**	une ligne dans la spec, puis relancer
.....	.....	.....

Les quatre classes de ce matin : **harness** (une vérification manquait) · **spec** (un trou) · **code** (il résiste) · **vraie erreur d'agent** (relancer, découper plus fin)

Puis rangez : `/exit` dans chaque session, et `git worktree list` · 15 minutes

On corrige la cause, pas la sortie.

Restitution

1

Ce qui a tourné **en parallèle**, et ce qui a dû attendre

2

Vos **tentations** d'intervenir, et leur classe

3

La **correction de harness** que vous emportez

4 ou 5 volontaires, 4 minutes chacun.

MCP

Brancher Claude sur un outil

Un serveur sur le catalogue de Tailspin, et le sous-agent qui s'en sert

Un serveur MCP, de près

```
server.registerTool('search_games', {
  description: 'Search the Tailspin Toys catalog. Case-insensitive'
    + ' text matched against title, category and publisher.',
  inputSchema: z.object({ query: z.string().trim().min(1) }),
  annotations: { readOnlyHint: true },
}, async ({ query }) => json(await searchGames(db, query)));

await server.connect(new StdioServerTransport());
```

Le transport

stdio : un processus que Claude Code lance · **HTTP** : une URL

La description

Elle décide quand Claude appelle l'outil, comme pour un skill

stdout

Il porte le protocole : un `console.log` casse le serveur

Sources : répétition du 15/10, mcp/catalog-server.ts écrit par Claude, extrait remis en forme · modelcontextprotocol.io, « Build an MCP server »

Le brancher

```
claude mcp add --scope project tailspin -- npx tsx mcp/catalog-server.ts
claude mcp list        # tailspin : ⏸ Pending approval
```

Portée	Rangé dans	Pour qui
`local`, par défaut	`~/.claude.json`	vous, sur ce projet
`project`	`.mcp.json`, versionné	l'équipe, après approbation
`user`	`~/.claude.json`	vous, sur tous vos projets

Ses outils : `mcp__tailspin__search_games`… · leurs noms chargés au démarrage, le reste **à la demande** · `mcp__tailspin__*` pour les autoriser

EXERCICE · ÉTAPE 1/4

Faites-le écrire

Terminal 3, à la racine, pas dans un worktree, sur une branche neuve (`git switch -c mcp-catalogue`) : `claude --model sonnet`, puis :

```
Écris un serveur MCP en stdio pour le catalogue, dans
mcp/catalog-server.ts, avec le SDK officiel v2
(@modelcontextprotocol/server, et zod). Deux outils en lecture
seule, qui réutilisent src/lib/ : search_games (un texte, cherché
dans le titre, la catégorie et l'éditeur) et get_game (par id).
La logique va dans src/lib/, testée par Vitest ; le serveur ne
fait que l'exposer. Ne lance pas d'agent.
```

Puis **lisez le serveur** : la description de chaque outil, et où partent ses messages d'erreur.

Projet perso : un serveur en lecture seule sur une donnée de votre projet. Fini en avance ? Lisez les tests : que prouvent-ils, et que ne prouvent-ils pas ?
App desktop : une session Local, à la racine, sans worktree ; d'abord, le bloc du cahier.

14 minutes

EXERCICE · ÉTAPE 2/4

Branchez-le

Dans Claude, `/exit` ; puis, dans le terminal :

```
claude mcp add --scope project tailspin -- npx tsx mcp/catalog-server.ts
claude --model sonnet
```

- Au lancement, « New MCP server found in this project » : le curseur est sur « Continue without… » ; choisissez **Use this MCP server**.
- Puis `/mcp` : **✔ tailspin · 2 tools** ; puis View tools : `search_games` et `get_game`, read-only.

Windows, « Connection closed » : `claude mcp remove tailspin -s project`, puis le même `claude mcp add`, avec `cmd /c` avant `npx`. Fini en avance ? `/context` : la ligne « MCP tools ».
App desktop : faites écrire `.mcp.json` par Claude (le cahier), puis Ctrl+N.

7 minutes

EXERCICE · ÉTAPE 3/4

Servez-vous-en

1. Demandez : « Quel est le jeu le mieux noté de chaque catégorie ? »
2. **Ctrl+O : par où est-il passé ?** Un appel à `tailspin`, ou autre chose ?
3. Puis : « Même question, en passant uniquement par les outils du serveur tailspin. »
4. Et hors catalogue : « Combien a levé Repo Rulers ? »

Fini en avance ? Une demande vague : « Un jeu pour ce soir ? » Que fait-il, sans consigne ?

App desktop : Ctrl+O fait défiler les vues ; la vue Verbose montre chaque appel d'outil.

9 minutes

EXERCICE · ÉTAPE 4/4

Confiez-le à un sous-agent

```
Crée le sous-agent concierge, dans .claude/agents/concierge.md :
il répond aux questions sur le catalogue en suivant les règles de
backlog/09-backer-concierge.md, sur Haiku, avec pour seuls outils
ceux du serveur tailspin. Déclare ce serveur dans son en-tête, en
ligne, avec la même commande que dans .mcp.json, puis retire-le
de .mcp.json : la conversation principale ne doit plus le voir.
Ne lance pas d'agent.
```

1. Relisez son en-tête : `model`, `tools`, `mcpServers`.
2. `/exit`, puis `claude --model sonnet` : un dossier `agents` neuf se charge au démarrage.
3. « Demande au concierge : quel jeu de stratégie est le mieux noté, et combien a-t-il levé ? »

Fini en avance ? `/context` : plus d'outil tailspin dans la conversation. Puis une demande vague au concierge.

App desktop : Ctrl+N, sur le même dossier, au lieu de `/exit` et de la relance.

15 minutes

MCP : quand, et à quel prix

Le besoin	La brique
Une CLI que Claude connaît déjà : `gh`, `az`, `dotnet`	un skill qui dit comment s'en servir
Un système hors de sa portée : Jira, Azure DevOps, une API interne	un serveur MCP, souvent publié par l'éditeur
Une interface étroite pour un agent : deux outils en lecture, pas de Bash	votre serveur, confié à un sous-agent

Un serveur stdio tourne sur votre poste, avec vos droits, hors du sandbox : on l'approuve comme une dépendance, et on se méfie de ceux qui lisent du contenu externe (injection de prompt).

Dans un repo, Claude a souvent un chemin plus court que votre serveur.

Sources : documentation Claude Code, page « MCP » · github.com/atlassian/atlassian-mcp-server · github.com/microsoft/azure-devops-mcp

Le coût humain

Ce qui disparaît

Surveiller chaque étape de l'agent, quand le harness vérifie à votre place

Ce qui reste

Lire le diff avant qu'il parte, le comprendre, et en répondre en production

+441 %

de durée médiane de l'étape de revue des PR, quand une équipe passe de sa plus faible à sa plus forte adoption de l'IA (Faros AI, 04/2026)

« Verification, not generation, is the real constraint on a factory. » — Addy Osmani

Sources : Addy Osmani, « Software Factories, Light and Dark », 07/2026 · Faros AI, « AI Engineering Report 2026 », 04/2026

Votre plan d'adoption à 30 jours

Semaine	Ce que je mets en place	Sur quel ticket de ma mission	Comment je sais que ça marche
1	le CLAUDE.md et la règle d'hier, commités, relus par l'équipe	le repo de la mission	la PR est relue et mergée
2	un hook et un skill partagés	le prochain ticket qui touche une migration	le skill se déclenche sans être nommé
3–4	une chaîne interview → spec → tickets, et une première parallélisation à deux sessions	le prochain ticket d'une demi-journée	le temps de relecture par PR

Écrivez la ligne 4 du cahier (votre correction de harness, votre première parallélisation), puis regroupez vos quatre lignes en un plan de quatre semaines : **un ticket de votre mission et un résultat mesurable par ligne**.

Mesurer sans se tromper

Mesurer l'activité

Le nombre de prompts, de sessions, de lignes générées

Mesurer le résultat

- le temps de relecture par PR
- la part des changements mergés du premier coup
- les retours sur la spec après le plan

+19 %

de temps avec l'IA, chez des développeurs qui pensaient avoir gagné 20 % (METR, 2025)

L'impression de vitesse ne suffit pas : mesurez sur vos propres tickets.

Auto-évaluation : les 6 objectifs

Objectif du programme	De 1 à 4
1. Piloter la fenêtre de contexte	
2. Rédiger des prompts précis	
3. Configurer son harness : CLAUDE.md, règles, hooks	
4. Concevoir des skills composables	
5. Composer des workflows spec-driven et choisir son niveau de supervision	
6. Paralléliser avec worktrees et sous-agents	

Dans le livret : **1** = je ne saurais pas le refaire seul ; **4** = je peux le montrer à mon équipe.

Vos questions

Ce qui reste ouvert après deux jours

Tour de table final

D'abord, vos attentes de jeudi matin : satisfaites ? en partie ? pas du tout ?

1

Une chose que vous **retenez** de ces deux jours

2

La **première brique** que vous posez lundi

3

Ce qui vous a **manqué** ou frustré

Une minute et demie chacun.

Ressources

Le récap

En ligne depuis 16h30 : à relire, avec les lectures et les sources

la page du livret,
jusqu'au 23 octobre

Les sources

La liste complète est dans le récap : documentation, articles, dépôts cités

Merci

Anthony Denneulin · Context Engineering, skills & workflows

linkedin.com/in/anthonydenneulin